

ROS STRUCTURE

ROBOTICS



POLITECNICO
MILANO 1863



Today schedule

- ROS Core components and structure *~ (what we expect inside middleware)*
- How to start ROS on your device (you should have at least one of these running):
 - Ubuntu 20.04
 - Distrobox
 - Docker
- ROS core components (practical) *→ all ROS tools + visualization*
- Debugging visualization tools

Note: we received applications for the challenges, we look into it during the weekend, expect news beginning of next week

ROS: ROBOT OPERATING SYSTEM



ROS main features:

- Distributed framework
- Reuse code
- Language independent → C++, python, ecc..
- Easy testing on Real Robot & Simulation
- Scaling

↳ ROBOTIC
middleware!

Standard Robotic
Simulator (Gazebo)

↑ usefull for
simulation

↓ or other tools of
(simulation)

ROS

ROS Components

- File system tools
- Building tools
- Packages → how to organize, compile ecc..
- Monitoring and GUIs
- Data Logging

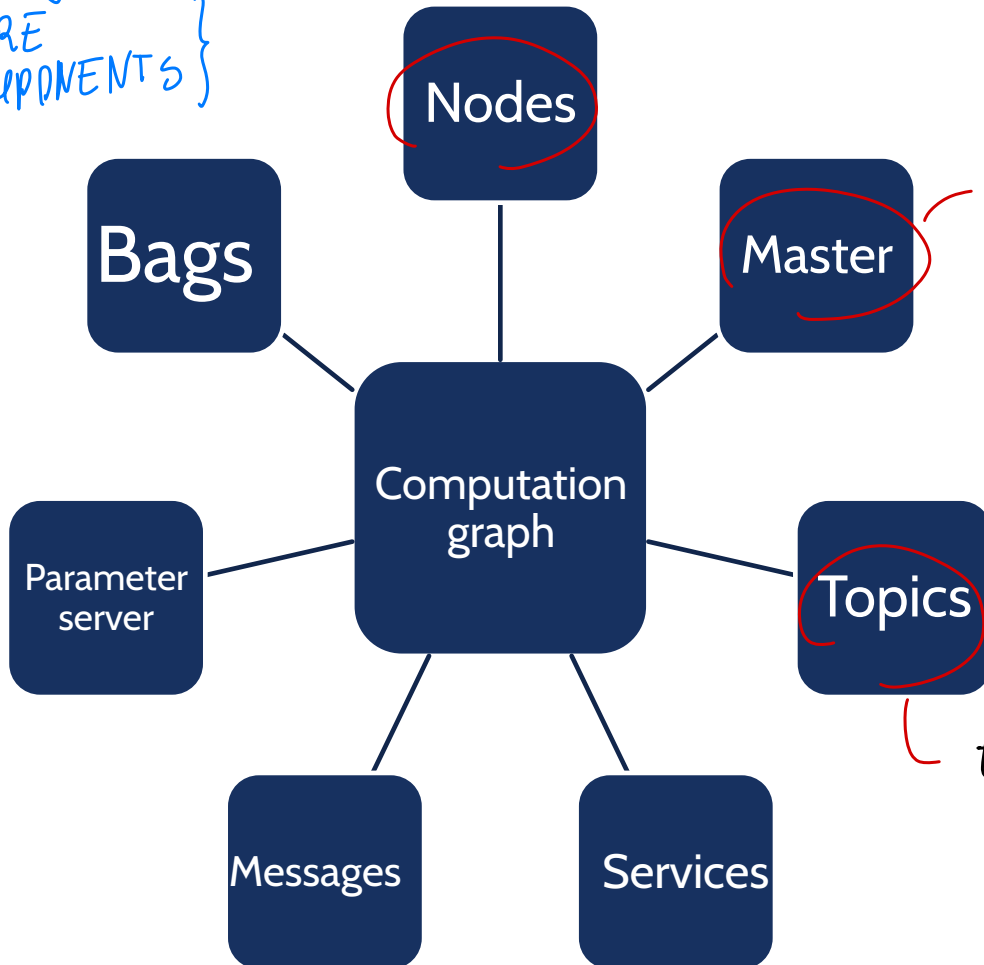
ROS based on
multiple
languages



ROS STRUCTURE: COMPUTATIONAL GRAPH



{ CORE COMPONENTS }



ROS is centralized,
one master communicate to
all other nodes

The *Computation Graph* is the
peer-to-peer network of ROS
processes that are processing data
together.

to communicate
between different
programs

NODES



↓ CORE components as "programm"

each node has a task assigned

Executable unit of ROS:

- Scripts for Python
- Compiled source code for C++

↑
nodes = different programm performing a task on Robot

Process that performs computation

`roslaunch package_name node_name`

Nodes exchange information via the graph

`roslaunch turtlesim turtlesim_node`

Meant to operate at fine-grained scale

A robot system is composed by various nodes

programm
doing
specific
task

Node 1) Take data from camera and send to ROS Net

Node 2) Take data from camera process and get info

Node 3) Take info, compute traj

Node 4) - -

MASTER

between multiple nodes you need a way to allow communication between them



↓ on ROS 1 you have a
MASTER which handles
information / communication

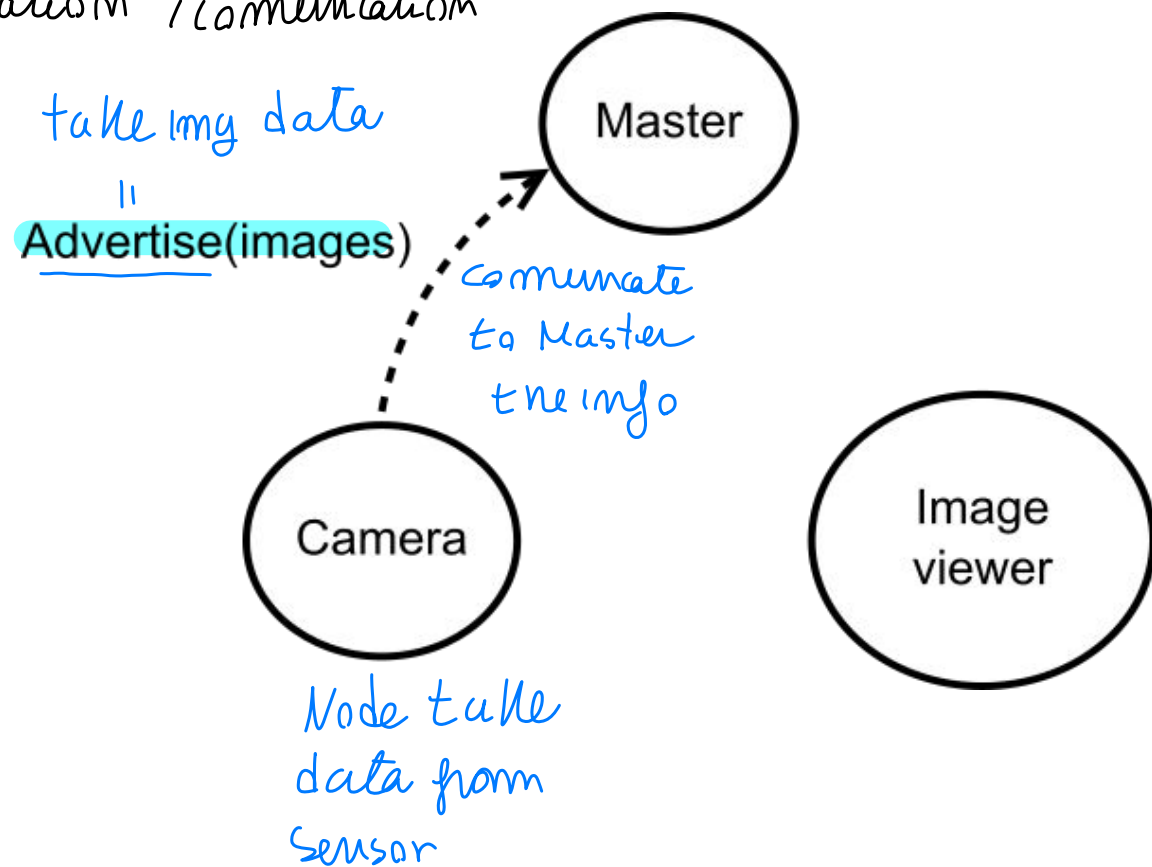
Provides naming and registration services

Essential for nodes interactions

One master for each system, even on distributed architectures

Enables individual ROS nodes to locate one another

One of the functionalities provided by roscore



MASTER



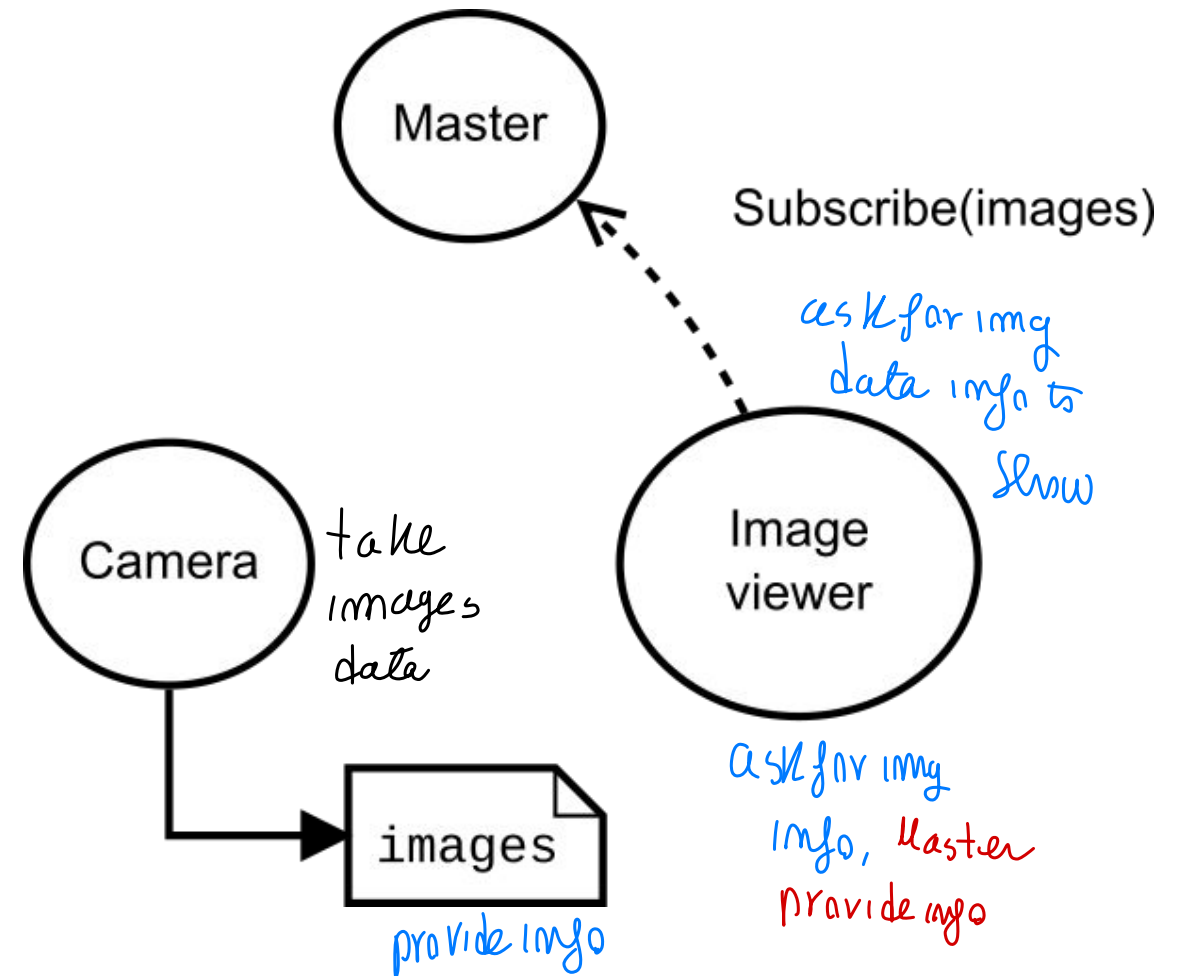
Provides naming and registration services

Essential for nodes interactions

One master for each system, even on distributed architectures

Enables individual ROS nodes to locate one another

One of the functionalities provided by roscore



MASTER



Provides naming and registration services

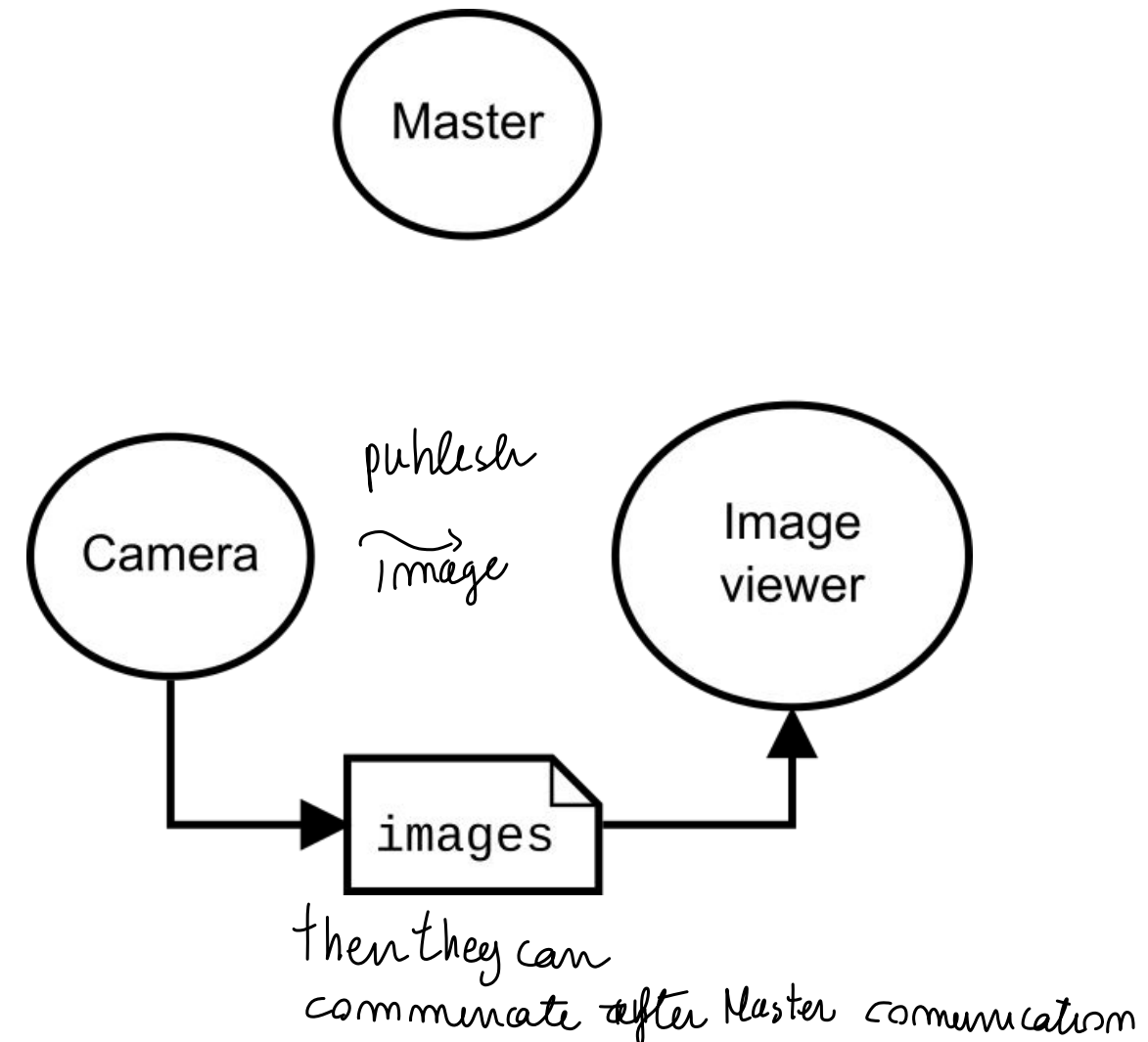
Essential for nodes interactions

One master for each system, even on distributed architectures

Enables individual ROS nodes to locate one another

One of the functionalities provided by roscore

*IF multiple camera you need
& way to differentiate the data going
to Ros-Net =>*



TOPICS



to differentiate ROS Data on the NET

↓
done by topics ⇒

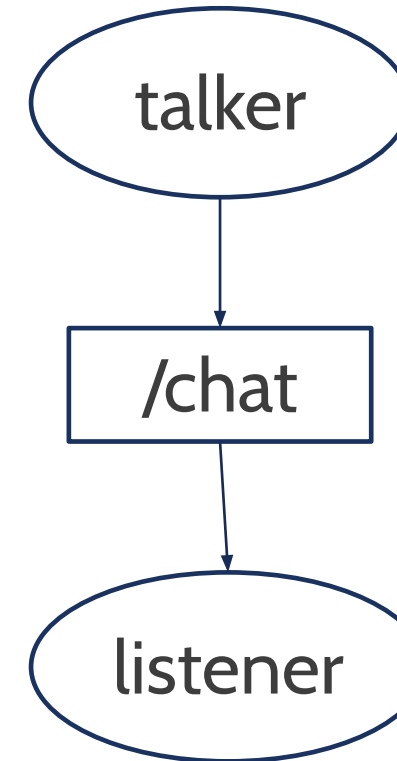
- **image** = type of info provided by the node
- **topic** is the name of the type of info published

↓
camera node publishes the image on a node

↓
and another node publishes on a new topic

↓
so you ask for a specific topic

} rear front camera



↑
create a connection on specific topic!

Named channels for communication

Implement the publish/subscribe paradigm

No guarantee of delivery

Have a specific message type

Multiple nodes can publish messages on a topic

Multiple nodes can read messages from a topic

TOPICS



Named channels for communication

Implement the publish/subscribe paradigm

No guarantee of delivery

Have a specific message type

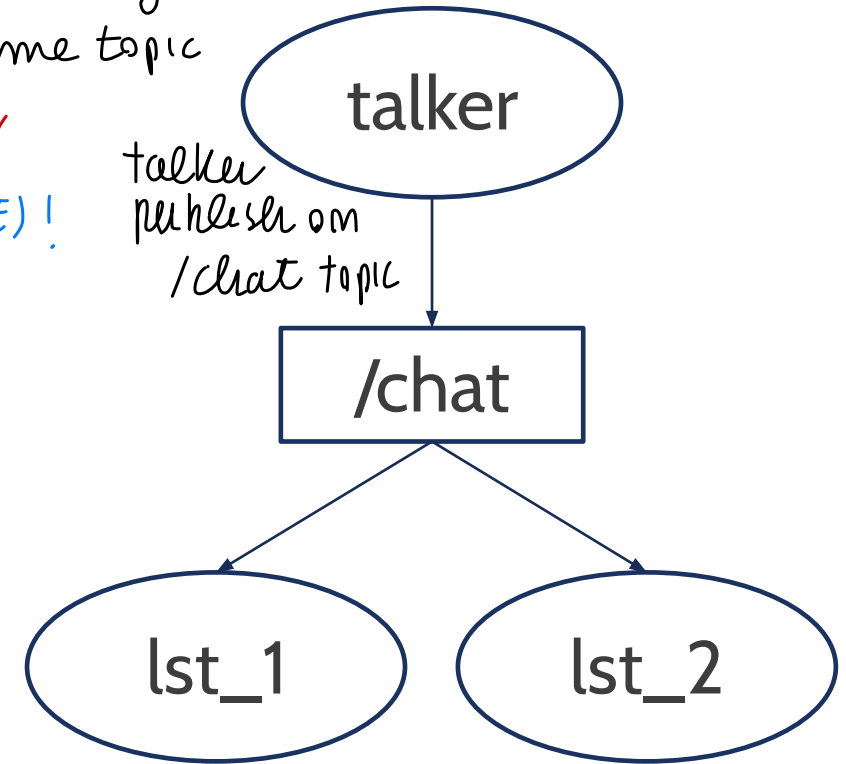
Multiple nodes can publish messages on a topic

Multiple nodes can read messages from a topic

you can have multiple
node subscribing to
the same topic



(POSSIBLE)!

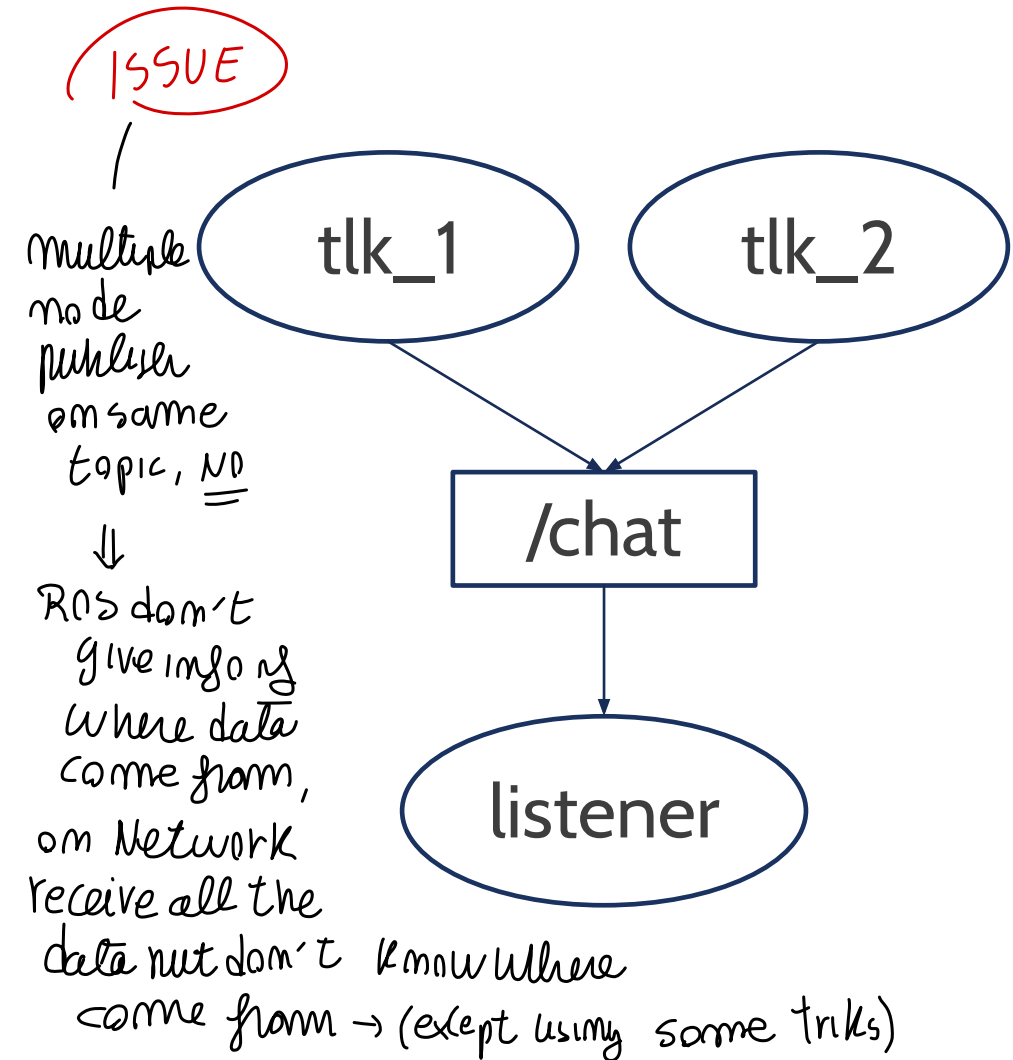


multiple node subscribe
to same topic → so they
receive same data

TOPICS



- Named channels for communication
- Implement the publish/subscribe paradigm
- No guarantee of delivery
- Have a specific message type
- Multiple nodes can publish messages on a topic
- Multiple nodes can read messages from a topic



MESSAGES

↪ modes publish info using messages on ROS NETWORK
⇒ ROS provide an extensive list of
standard message you can publish

↓
usually publish info by standard
ROS message...

Messages are exchanged on topics

They define the type of the topic

Various already available messages

It is possible to define new messages using a
simple language

Existing message types can be used in new
messages together with base types

std_msgs/Header.msgs

uint32 seq
time stamp
string frame_id

std_msgs/String.msg

string data

sensor_msgs/Joy.msg

std_msgs/Header header
float32[] axes
int32[] buttons



MESSAGES



Common messages

Messages are exchanged on topics

They define the type of the topic

Various already available messages

It is possible to define new messages using a simple language

Existing message types can be used in new messages together with base types

for a complete new
sensor msg defined

You can create **CUSTOM MESSAGES** ~>

Quick recap:

14 base types

32 **std_msgs** (standard)

29 **geometry_msgs** → for planning
& control
(motion)

26 **sensor_msgs** →

sensor data
(img, points...)

...and more

specific scenarios

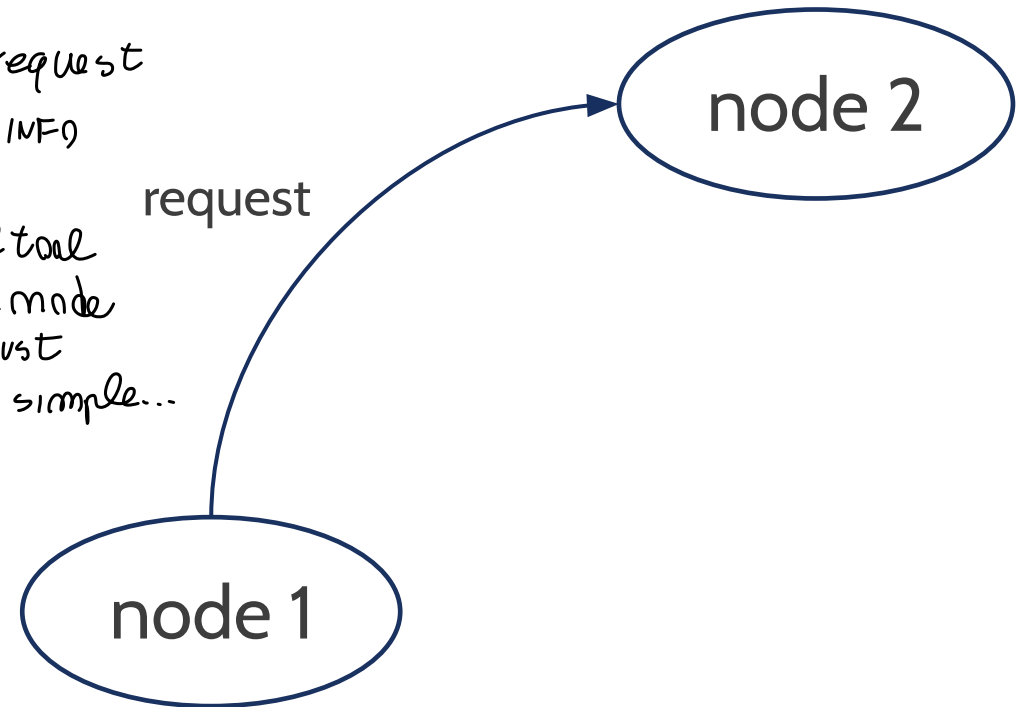
structure usable, particular structure
(Meffers standards!) → for reusability

SERVICES

similar to messages!

↓ more advanced kind of messages used to
interrogate the node for a specific INFO
↳ node 1 ask INFO to node 2
by standard message request
and node 2 return INFO

↓
Usefull tool
when a node
require just
one INFO simple...



Work like remote function calls

Implement the client/server paradigm

Code waits for service call to complete

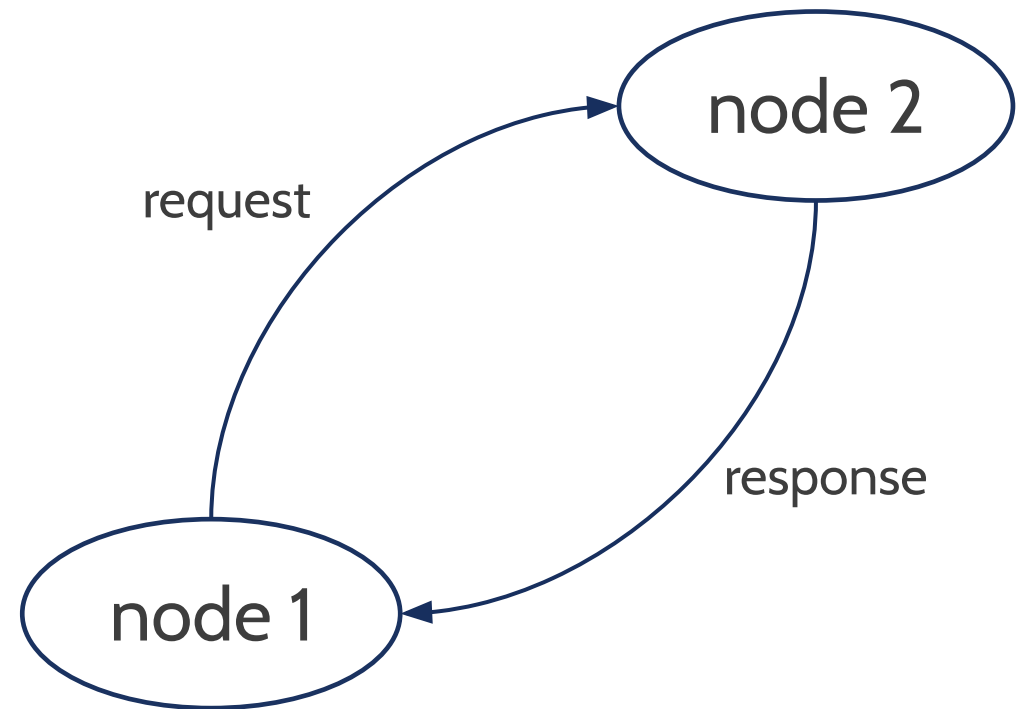
Guarantee of execution



SERVICES



Work like remote function calls
Implement the client/server paradigm
Code waits for service call to complete
Guarantee of execution



SERVICES



how it is structured

Work like remote function calls

Implement the client/server paradigm

Code waits for service call to complete

Guarantee of execution

Use of message structures

example/AddTwoInt.srv

int64 A

int64 B

int64 Sum

} request

} response

PARAMETER SERVER

component relevant



we mostly write nodes (C++ based), so the parameters can be written in code and & change, recompile all → ROS PARAMETER SERVER

Shared, multivariable dictionary that is accessible via network

Nodes use this server to store and retrieve parameters at runtime

Not designed for performance, not for data exchange

Connected to the master, one of the functionalities provided by roscore

↓
store some param. to use it inside ROS, so ask for a specific value @ START

name	value
/gains/P	10.0
/gains/I	1.0
/gains/D	0.1
use_sim_time	True

rosparm [set|get] name value

rosparm set use_sim_time True

rosparm get use_sim_time

> True

} PID data accessible @ START
↓
you don't need to recompile all each time, just this component

PARAMETER SERVER



some available type on PARAM SER

Shared, multivariable dictionary that is accessible via network

Nodes use this server to store and retrieve parameters at runtime

Not designed for performance, not for data exchange

Connected to the master, one of the functionalities provided by roscore

Available types:

32-bit integers → number

Booleans

Strings

• Doubles

• ISO8601 dates

Lists

Base64-encoded binary data

BAGS

usefull ROS tool

BAG file $\equiv$ way to record data inside
ROS $\rightarrow$ how data are given



File format (*.bag) for storing and playing back messages

Primary mechanism for data logging

Can record anything exchanged on the ROS graph (messages, services, parameters, actions)

Important tool for analyzing, storing, visualizing data and testing algorithms.

$\downarrow$ standard way to record what happens inside
ROS Net $\rightarrow$ dummy TESTING, to analyze
test result later $\Rightarrow$ using same
input given as ".bag"

rosv bag record -a

rosv bag record /topic1 /topic2

rosv bag play ~/bags/fancy_log.bag

rqt_bag ~/bags/fancy_log.bag

recall sensor data and save
that data to test & selection

"bag" $\rightarrow$ store all info
 $\hookrightarrow$ message + when they appear (timestamp)

ROSCORE

FIRST command running each time you start ROS



start the needed components

roscore is a collection of nodes and programs that are pre-requisites of a ROS-based system

↳ run → MASTER / PARAM.SERVICE..

Must be running in order for ROS nodes to communicate

runned
by ROSCORE

Launched using the roscore command.

Elements of roscore:

- a ROS Master
- a ROS Parameter Server
- a roscout logging node

A RECAP ON HOW TO INSTALL

ROBOTICS



POLITECNICO
MILANO 1863

INSTALLATION



If you have Ubuntu/Debian:

- <http://wiki.ros.org/noetic/Installation/Ubuntu>
- Install ros-noetic-desktop-full

Test it:

- new terminal -> `roscore`



INSTALLATION (distrobox)

If you have other linux distro/Ubuntu version:

Install Docker: <https://docs.docker.com/engine/install/>

Install Distrobox: <https://github.com/89luca89/distrobox#installation>

Then:

- `distrobox create robotics -i smentasti/robotics:latest`
- `distrobox enter robotics`

You can edit your `.bashrc` file to load variables only inside distrobox

You can configure a different terminal program to automatically load distrobox

Example terminator config provided (edit `~/.config/terminator/config`)



INSTALLATION (distrobox)

If you have issue you can also try with a more standard image:

- `distrobox create robotics -i ubuntu:20.04`
- `distrobox enter robotics`

Test it:

- `new terminal -> distrobox enter robotics`
- `source /opt/ros/noetic/setup.bash`
- `roscore`

This have to work, than you can have more advanced setup (e.g., `.bashrc` edit, terminal program to automatically load the environment, etc.)



INSTALLATION (Docker)

First install docker:

- <https://docs.docker.com/engine/install/>
- Build an image using a Dockerfile (provided, it should have most of the required packages, run the command in the Dockerfile folder):
 - `docker build -t robotics:stable .`
- install docker compose:
 - `sudo apt install docker-compose`

Test it

- new terminal -> `docker-compose -f robotics.yaml up`
- new terminal ->
 - `docker exec -it --user your_username robotics_app /bin/bash`
 - `source /opt/ros/noetic/setup.bash`
 - `roscore`

↘ to mount image

FIRST terminal
instruction to
↓ execute docker

1) `docker build -t robotics:stable` $\rightarrow$ create `robotics:stable` image
`docker image ls` $\rightarrow$ to show "robotics stable"

2) start the image to use it (36 min)

$\rightarrow$ on ubuntu on / robotics/Docker

$\downarrow$

mounting on `robotics.yuml` $\rightarrow$ change `NAME`

and on volumes:



WINDOWS USERS CAN USE DOCKER

If you only have windows you can still docker on the linux subsystem

- Install docker: <https://docs.docker.com/desktop/install/windows-install/>
- It ask you to install linux subsystem if not installed
- Same commands and procedure as other docker scenarios
- Suggest to create a robotics folder in the Documents folder and mount it in docker:
- `docker run --net=ros --env="DISPLAY=novnc:0.0" -it -v C:\Users\Simone\Documents\Robotics:/home/simone/robtics robotics /bin/bash`

Gui at: <http://localhost:8080/vnc.html> with the vnc container running



TMUX Commands recap (if you work from terminal)

- | | |
|-------------------------------------|------------------------------|
| - tmux new -s session_name | create a new session |
| - ctrl+b -> % | split vertically |
| - ctrl+b -> “ | split horizontally |
| - ctrl+b -> arrow | move to a different terminal |
| - ctrl+b -> d | exit session |
| - tmux a | attach to last session |
| - tmux a -t session_name | attach to session |
| - tmux kill-session -t session_name | kill session |

While using ROS you need multiple terminals...

↳ doing multiple things...

you need to run multiple terminal windows on

some Docker ↴

"tmux" to create virtual terminal

than... inside the docker

> roscore to start running ... occupy one terminal,

so you use `tmux new -s "session name"` → virtual terminal
running inside

`CTRL + b + %` to split terminal

← the docker img

`CTRL + b + arrow` to move inside terminals

→ ↑
← ↓

We can do multiple action on different terminal docker virtual

→ but still running

OR kill the virtual session

⇒ usefull to execute multiple
terminal on same terminal

↳ last usefull component is a GUI when we
want a user interface

the "vnc.sh" file
script start docker
container which run

vnc file... `vnc "vnc.sh"` input

↓
← { done by vnc running
on custom network

VmC inside docker ~ VmC running



VmC server running on

http://localhost:8080/vnc.html

GUI of what
you run

(Ubuntu) open terminal, start roscore



nano .bashrc

```
source /opt/ros/noetic/setup.bash ]; then
source /repos/orb_slam/devel/setup.bash --extend
fi
if [ "$HOSTNAME" = robotics.simone-xmg ]; then
source /opt/ros/noetic/setup.bash
source /home/simone/robotics/devel/setup.bash
fi
```

similar by destrabox ...

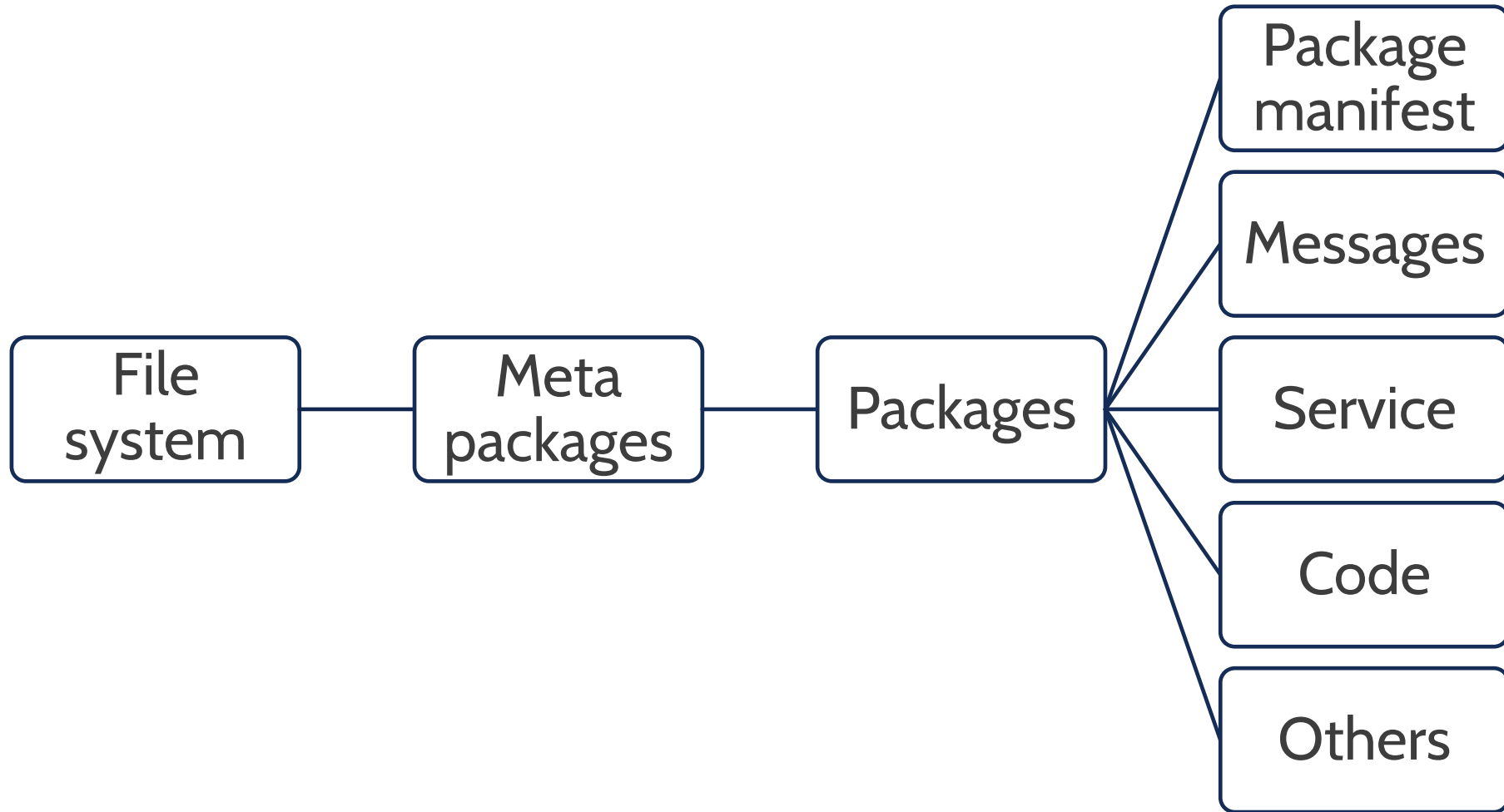
ROS FILESYSTEM

ROBOTICS

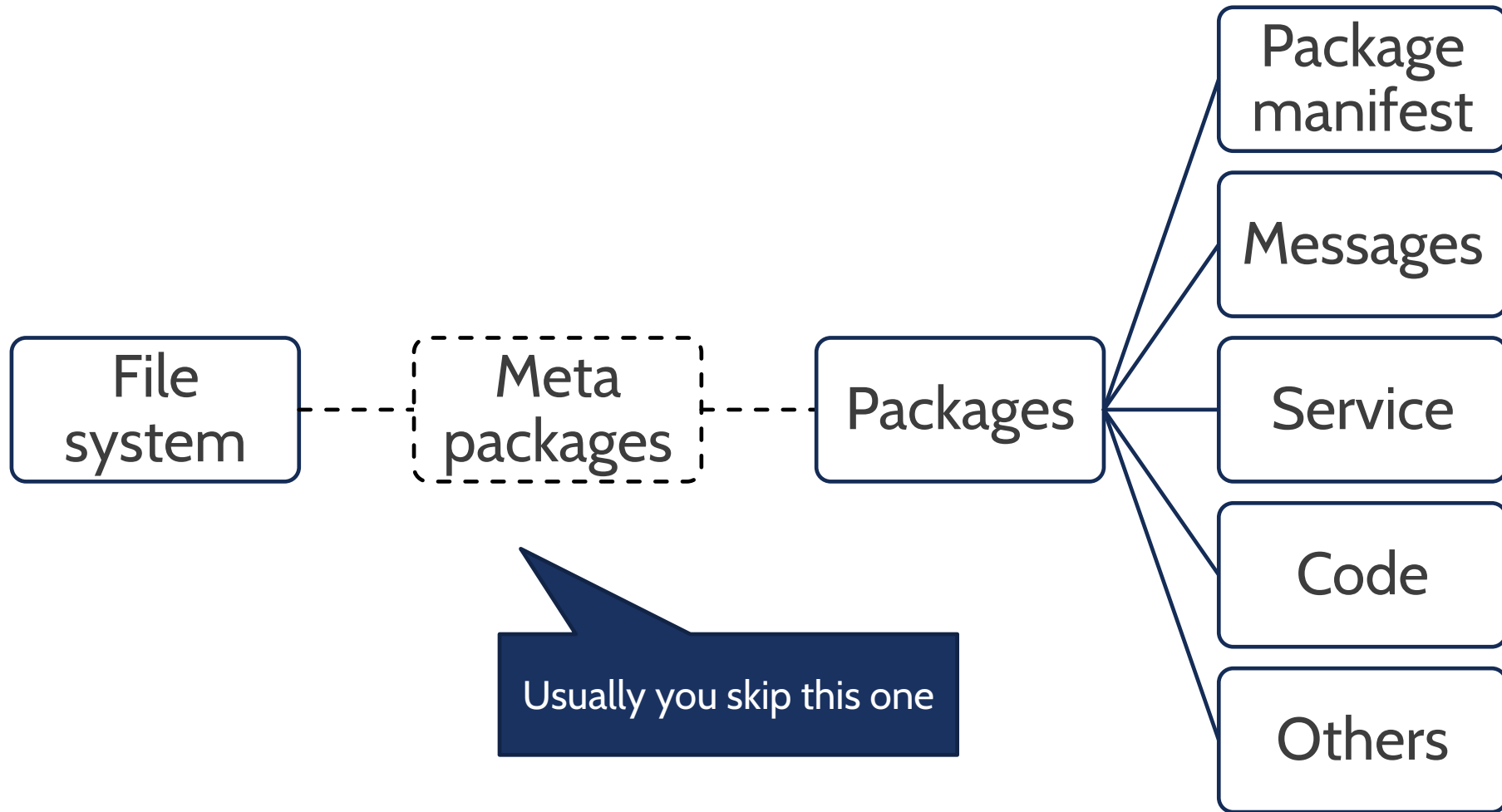


POLITECNICO
MILANO 1863

ROS FILE SYSTEM



ROS FILE SYSTEM



PACKAGES AND METAPACKAGES



PACKAGES

- Atomic element of ROS file system
- Used as a reference for most ROS commands
- Contains nodes, messages and services
- package.xml used to describe the package
- Mandatory container

METAPACKAGES

- Aggregation of logical related elements
- Not used when navigating the ROS file system
- Contains other packages
- package.xml used to describe the package
- Not required



STRUCTURE OF A PACKAGE

Folder structure:

/src, /include, /scripts (coding)

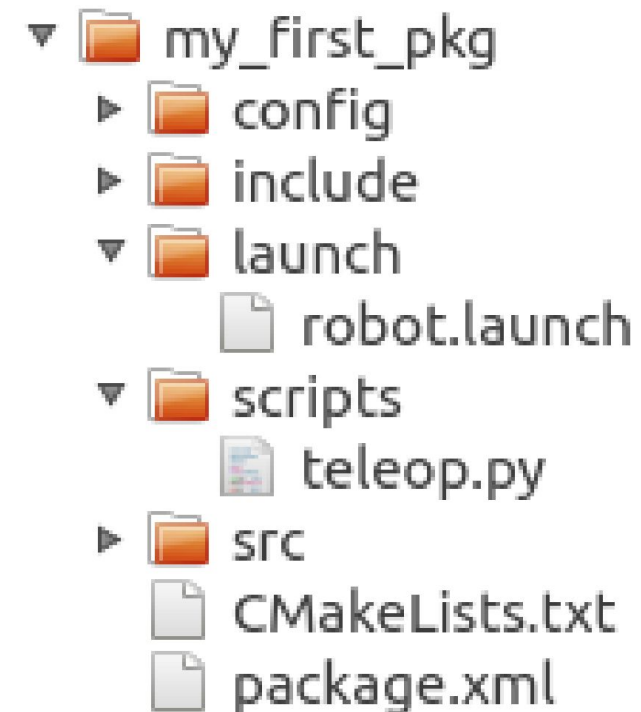
/launch (launch files)

/config (configuration files)

Required files:

CMakeList.txt: Build rules for catkin

package.xml: Metadata for ROS



ROS COMMANDS

ROBOTICS



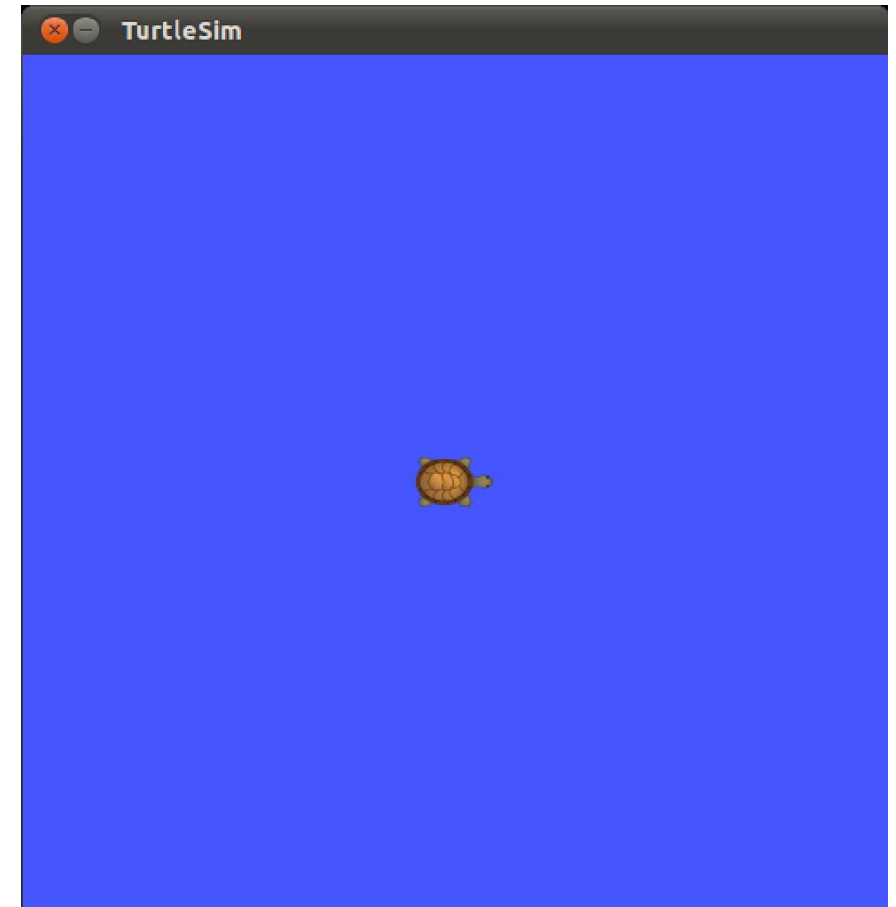
POLITECNICO
MILANO 1863

FILE SYSTEM TOOLS



Ros Desktop-full come with lots of tutorials and tools

Before creating our own package and start writing some code we will learn how to navigate the ROS file system and use the turtlesim package to test some of the most useful tools





FILE SYSTEM TOOLS

Change directory in the ROS file system

roscd [package_name[/subdir]]

roscd roscpp && pwd /opt/ros/kinetic/share/roscpp

roscd roscpp/srv /opt/ros/kinetic/share/roscpp/srv

roscd robby_roboto ~/catkin_ws/src/robby_roboto

FILE SYSTEM TOOLS



Getting information about installed packages

rospack <subcommand> [options] [package]

subcommands (among the others)

depends [package] package dependencies

find [package] find package directory

list list available packages

profile scan all workspace and index packages

rospack find roscpp /opt/ros/kinetic/share/roscpp

rospack list <several packages>



STARTING THE MIDDLEWARE

To start the ROS middleware just type in a terminal
roscore

Now it is possible to display information about the elements currently running

`roscnode list` $\leadsto$ show what is happening inside

`rostopic list` $\leadsto$ topic of the program published

`rostopic echo /rosout`

`rosservice list`

`rqt_graph`



DEALING WITH NODES

Getting information about running nodes

roscall <command> [other_commands]

subcommands (among the others)

ping test connectivity to node

info print information about node

kill kill a running node

cleanup purge registration information of unreachable nodes

roscall list

roscall info /rosout

STARTING ROS NODES



To start a ROS node type in a terminal

roslaunch [package_name] [node_name]
where node exist node name

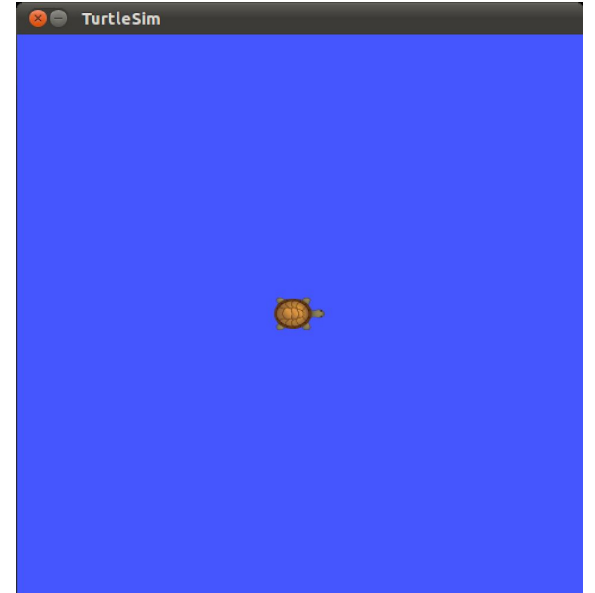
roslaunch turtlesim turtlesim_node *→ we see some numbers*

rostopic ping /turtlesim

rostopic info /turtlesim

rostopic list:
we see what is happening

↓
on GUI ⇒
we see what is happening inside



/turtlesim

STARTING ROS NODES



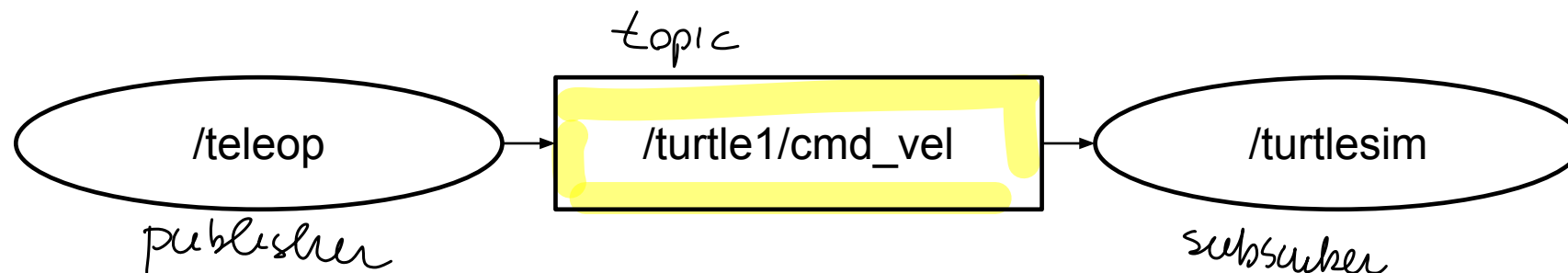
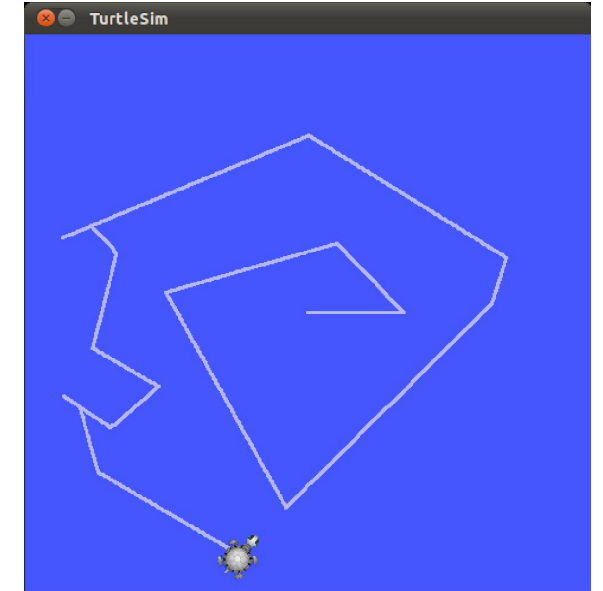
In a new terminal

```
roslaunch turtlesim turtle_teleop_key
```

Notes:

`turtle_teleop_key` is publishing the key strokes on a topic

`turtlesim` subscribes to the same topic to receive the key strokes



"Terminal"

ros mode info /mode-name

contains a lot of info

{ INFO about our mode }

But ROS can run on multiple devices, so the IP address can be very useful to pass info

Services you can call on that topic

all run locally on your device

you can change transport protocol

```
TurtleSim
alessandro@alessandro-MacBookPro:~$ roslaunch turtlesim turtlesim.launch
[ INFO ] [1679831755.881717676]: Starting turtlesim with pid: 6614

alessandro@alessandro-MacBookPro:~$ rosnode info /turtlesim
Node [/turtlesim]
  Publications:
    * /rosout [rosgraph_msgs/Log]
    * /turtle1/color_sensor [turtlesim/Color]
    * /turtle1/pose [turtlesim/Pose]
  Subscriptions:
    * /turtle1/cmd_vel [unknown type]
  Services:
    * /clear
    * /kill
    * /reset
    * /spawn
    * /turtle1/set_pen
    * /turtle1/teleport_absolute
    * /turtle1/teleport_relative
    * /turtlesim/get_loggers
    * /turtlesim/set_logger_level
  contacting node http://alessandro-MacBookPro:35703/ ...
  Pid: 6614
  Connections:
    * topic: /rosout
      * to: /rosout
      * direction: outbound (40863 - 127.0.0.1:50446) [29]
      * transport: TCPROS
alessandro@alessandro-MacBookPro:~$
```

which topic this mode is publishing (useful to check the topic published)

mode sub ... → you have to provide info as [turtle1/cmd_vel]

↑ topic type known...

where the mode is running

this mode publishes some topics!

```
alessandro@alessandro-MacBookPro:~$ rostopic list
/rosout
/rosout_agg
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
alessandro@alessandro-MacBookPro:~$
```

topic published by turtle mode

subscribing mode → existing topic

```
/turtle1/pose
alessandro@alessandro-MacBookPro:~$ rostopic info /turtle1/pose
Type: turtlesim/Pose
Publishers:
  * /turtlesim (http://alessandro-MacBookPro:35703/)
Subscribers: None
```

↓

we can subscribe to it by "echo"

→ rostopic echo /turtle1/pose → get info

constantly publishing mode!



to check freq. of publish of topic `rostopic hz topic_name`
tells you the freq of update publish data

```
simone@robotics:~$ rosrun turtlesim turtlesim
simone@robotics:~$ rostopic info /turtle/pose
Type: turtlesim/Pose
Publishers:
 * /turtlesim (http://127.0.0.1:36375/)
Subscribers:
 * /rostopic_005_1678883515399 (http://127.0.0.1:43029/)
simone@robotics:~$
```

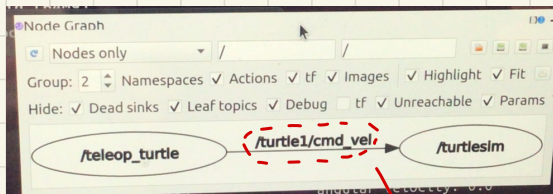
now we
are subscribing
on this mode

→ rosmode on network
subscribing on the topic → local on
machine

we can analyze in this way what is happening inside ros network



there is a better way to check what is happening on ROS network!
by "rqt-graph"



(simple
visualization)

ROS tool usefull for
net analysis

topic published
by /teleop-turtle
to → /turtlesim

• To control the
turtle instead of using turtle controller...

→ sometimes

for DEBUGGING you wanna publish topic command by
cmdline

→ by command ..

```
simone@robotics:~$ rostopic pub /turtle1/cmd_vel geometry_msgs/Twist -r 1 -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, 1.8]'
```

topic to publish

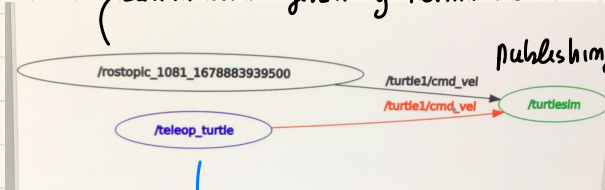
type of
Data
message

rate
@ which publisher
(1 Hz)

Data to publish

↓ Linear
angular
Velocity

command given by terminal



key boards
control publishing

+ usefull tools for DATA VISUALIZATION

↓

too much number hard to check & debug

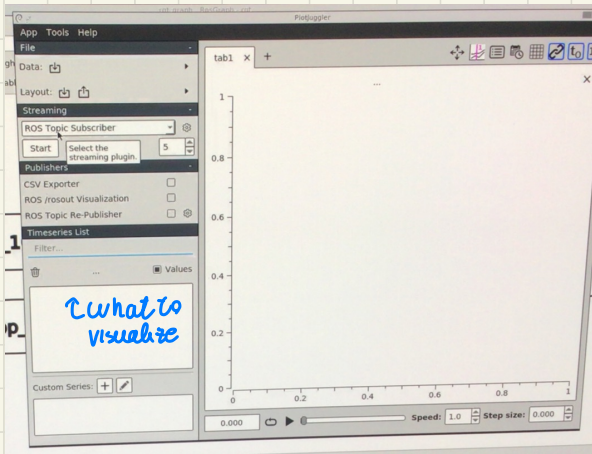
↓

by » rosrn plot Juggler plot Juggler

Which is usefull for numeric data visualization

(solve issue
here!)

"ROS Topic Subscriber"
don't see ammine cmd!



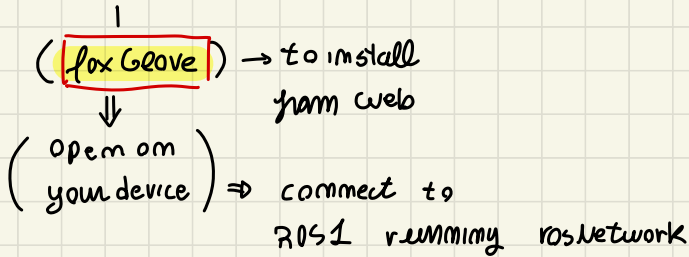
Usefull to visualize
when working
with odometry /
position...

↓
What is happening?!

(to see multiple
updates)

another good numeric data visualization
are others...

foxgloves to see more data visualization



DOWNLOAD some
code environment

» catkin_make to create an environment where
you can put all code
↑
(compile environment)

↳ or /src we will put the code

DEALING WITH TOPICS



To show the running node type in a terminal

```
rqt_graph
```

To plot published data on a topic

```
rqt_plot /turtle1/pose/x /turtle1/pose/y
```

```
rqt_plot /turtle1/pose/x:y
```

To monitor a topic on a terminal type

```
rostopic echo /turtle1/cmd_vel
```

DEALING WITH TOPICS CONT.



Getting information about ROS topics

rostopic <command> [topic_name]

subcommands (among the others)

echo print messages to screen

find find topics by type

hz display publishing rate of topic

info print information about active topic

list list active topics

pub publish data to topic

type print topic type



DEALING WITH TOPICS CONT.

Getting **information about ROS topics**

```
rostopic type [topic_name]
```

```
rostopic type /turtle1/cmd_vel
```

Publishing ROS topics

```
rostopic pub [topic] [msg type] [args]
```

```
$ rostopic pub my_topic std_msgs/String "hello there"
```

```
$ rostopic pub -r 10 /cmd_vel geometry_msgs/Twist '{linear: {x: 2.0, y: 0.0, z: 0.0},  
angular: {x: 0.0,y: 0.0,z: 0.0}}'
```



DEALING WITH TOPICS CONT.

```
$ rostopic pub -1 /turtle1/cmd_vel geometry_msgs/Twist -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, 1.8]'
```

The **-1** option **force rostopic to publish the message only once**, if you want to publish the message at a specific frequency you will use:

```
$ rostopic pub /turtle1/cmd_vel geometry_msgs/Twist -r 1 -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, 1.8]'
```

Where the **-r 1** option **specify that the message will be published at 1hz frequency**



MESSAGES (ALSO SERVICES)

Getting information about msg/srv files

rosmmsg <command> [msg/srv_file]

subcommands (among the others)

- | | |
|-----------------|---|
| show | Display the fields in the msg/srv. |
| list | Display names of all msg/srv. |
| package | List all the msg/srv in a package. |
| packages | List all packages containing the msg/srv. |

rosmmsg show Pose

rosmmsg package nav_msgs



DEALING WITH SERVICES

Calling services from command line and getting information:

rosservice <command> [other_commands]

subcommand (among the others)

list	Print information about active services.
node	Print name of node providing a service.
call	Call the service with the given args.
args	List the arguments of a service.
type	Print the service type.
find	Find services by service type

rosservice call /reset

rosservice type /reset

BAGS



bag: file format to store messages data

Used to test different algorithm with the exact same input and to debug a system when it's not monitorable at runtime

To record a bag use:

`rosbag record`

to record all the topics use:

`$ rosbag record -a`

to record only a subset of the topic use:

`$ rosbag record topic1 topic2 etc`

BAGS



To get info regarding a beg use the command:

```
$ rosbag info bag_name
```

To play a bag run:

```
$ rosbag play bag_name
```

remember that **to run rosbag you need an active ros session** (roscore should be on)

Always monitor your bag size, sometimes logging all the topics (if you are working with cameras) is not the best idea because you will produce more data/sec than your max disk writing speed.

CREATE THE ROS WORKSPACE

ROBOTICS



POLITECNICO
MILANO 1863



CREATING THE WORKSPACE

ROS uses a custom compiling environment called **Catkin**

cmake/make with specific flags

Requires a workspace with a specific structure

Easy to setup and “easy” to use

```
mkdir -p ~/robotics/src
```

```
cd ~/robotics/
```

```
catkin_make
```

```
echo "source ~/robotics/devel/setup.bash" >> ~/.bashrc
```

```
source ~/.bashrc
```

WORKSPACE STRUCTURE



Source space (/src):

contains the source code of catkin packages.

Subfolder of this are the ROS packages you want to add to your system

All your stuff goes here!

Build space (/build):

space where cmake is invoked to build the catkin packages

cmake and catkin keep their cache information and other intermediate files here

Not where
catkin_make
is invoked!

Devel space (/devel):

Space where built targets are placed prior to being installed

PACKAGE CREATION



Command to create a new package

```
catkin_create_pkg [package_name] [depend1] [depend2] [depend3]
```

Before running the script `cd` to your src directory, then:

```
catkin_create_pkg pub_sub std_msgs rospy roscpp
```

Important Notes

roscpp and rospy are client libraries to use C++ and Python

!!Before being able to do that you should have created a ros_workspace!!

PACKAGE CREATION



cd to the new package, the **script should have created:**

- CMakeLists.txt
- package.xml
- include folder
- src folder

cd to your catkin workspace root to compile the new package, simply using **catkin_make**

EDITORS/ IDEs

ROBOTICS



POLITECNICO
MILANO 1863

ROSED



rosed is part of the **rosbash** suite

Allow the user to **edit files using directly the package name**, rather than typing the entire path

```
rosed [package_name] [filename]
```

```
rosed roscpp Logger.msg
```

The default editor is vim

You can edit the .bashrc file setting a more user friendly editor

IDEs



No official IDE by ROS

C++ editor with ROS specific plugins

On ROS wiki you can find guides on how to properly configure the plugins

<http://wiki.ros.org/IDEs>

Simply add some features like easier compiling and some debug tools



Roboware



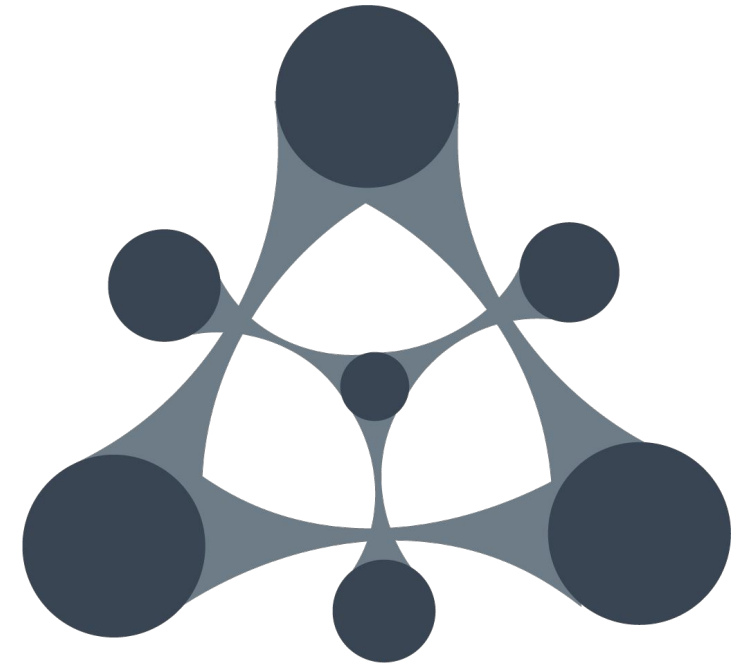
Based on Visual Studio

Designed for ROS

No need to install third parties plugin

Offers some functionalities:

- Run program directly inside Roboware
- Debugger
- Automatic file generation
- CMakeLists and Package.xml automatic update (partial)
- Integrated ros tool



ROS DEVELOPMENT

ROBOTICS



POLITECNICO
MILANO 1863

EVERYTHING HAPPENS IN NODES

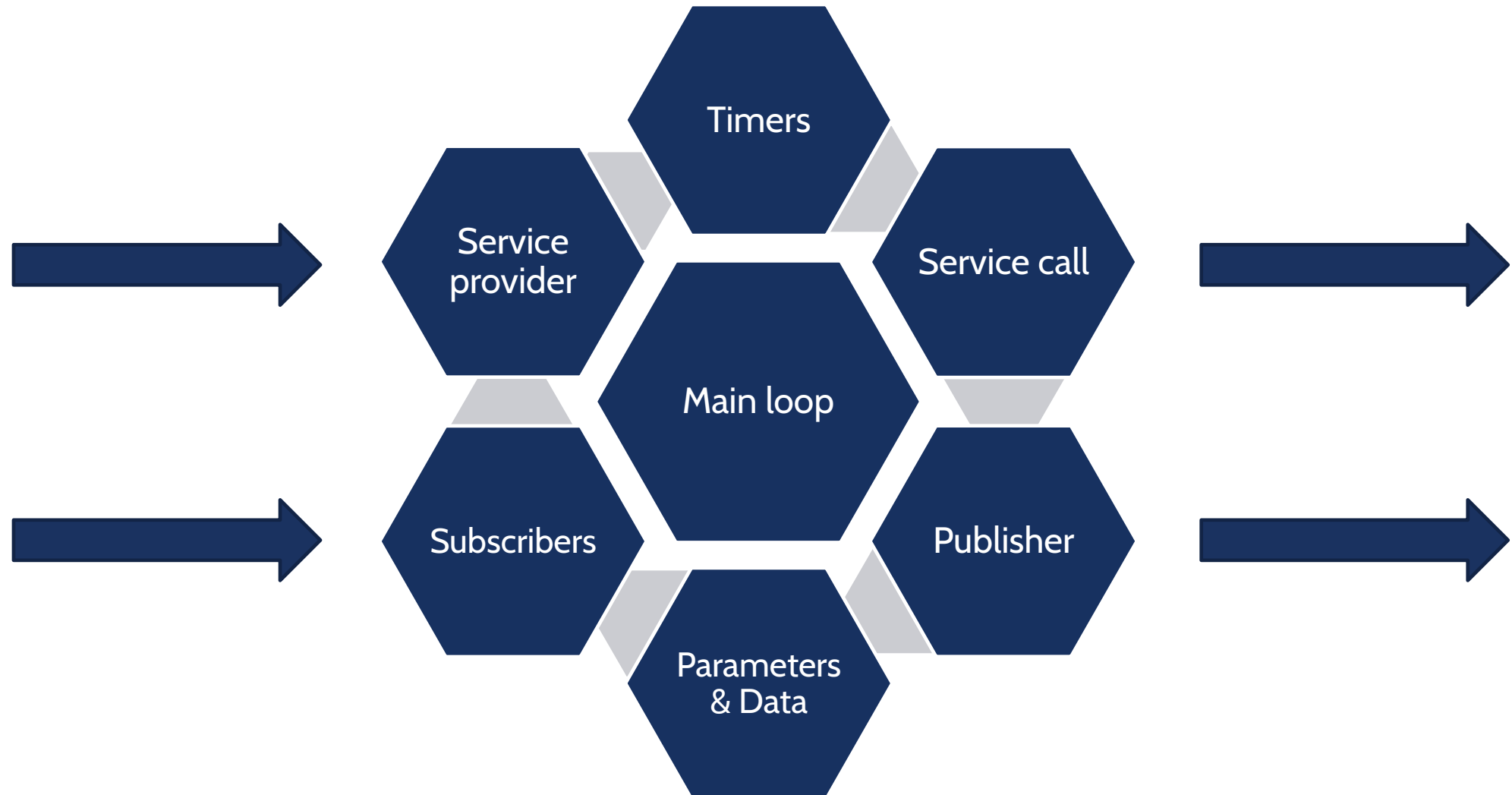


Nodes are the main and atomic element of ROS. Each node is an independent process.



How do we create a node?
Write code in C++ or Python

INSIDE THE NODE





INITIALIZATION

Any node has to be registered to the ROS master using an unique identifier

The actual node is initialized using an handler

Each executable has an **unique name**

Each executable may have multiple handlers

```
void ros::init(argv, argc, std::string node_name, uint32_t options);  
ros::init(argc, argv, "my_node_name");  
ros::init(argc, argv, "my_node_name", ros::init_options::AnonymousName);  
  
ros::NodeHandle nh;
```



MAIN LOOP

Each ROS node loops waiting for something to do

At each loop checks:

is there a message waiting to be received?

is there a completed timer?

is there a parameter to be reconfigured?

Two ways to implement the main loop:

- Automatically, no developer intervention
- Manual, specific sleep time and execution at each loop

```
ros::spin();
```

```
ros::Rate r(10); //10 hz
```

```
while (ros::ok()) {
```

```
    /* some execution */
```

```
    ros::spinOnce();
```

```
    r.sleep();
```

```
}
```

PARAMETERS



Stored in the parameter server and retrieved at the beginning of the execution

Adjustable at runtime using dynamic reconfigure

Global parameters and relative parameters (in the node namespace)

```
if(!nh.getParam("/global_name", global_name)) { /* :( */ }  
if(!nh.getParam("relative_name", relative_name)) { /* :( */ }  
nh.param<std::string>("param_name", default_param, "default_value");
```

PUBLISHER



Used to publish messages on a ROS topic

On declaration connect the publisher to a topic and define the type of the message

Can be called from everywhere

The frequency of the messages are not set

```
ros::Publisher pub = nh.advertise<std_msgs::String>("topic_name", 5);  
std_msgs::String str;  
str.data = "hello world";  
pub.publish(str);
```


SUBSCRIBER



Used to read messages from a ROS topic

On declaration connect the subscriber to a topic and define the type of the message

Call a specific function when receive a message

Operate at a given frequency

```
ros::Subscriber sub = nh.subscribe("topic_name", 10, callback);
```

```
void [class::]callback(const pack_name::msg_type::ConstPtr& msg)
```

TIMER



Used to execute something after a specific time (repeatable)

When the timer ends a callback function get called

Tied to ROS internal clock

```
ros::Timer timer = nh.createTimer(ros::Duration(0.5), callback);
```

```
void [class::]callback(const ros::TimerEvent& t)
```



SERVICE PROVIDER (SERVER)

Answer to a service call and execute some logic associated with the content of the call

On declaration connect to the callback with the implemented logic

The answer of the service is already in the callback

```
ros::ServiceServer s = nh.advertiseService("service", callback);
```

```
bool [class::]callback(pack::srv_type::Request& req, pack::srv_type::Response& res);
```



SERVICE PROVIDER (SERVER)

Generates the call for a specific service

On declaration is connected to the a service identified by a name

Can be called everywhere in the code

May result in a bad call

```
ros::ServiceClient cl = nh.serviceClient<pack::srv_type>("service");  
pack::srv_type srv;  
/* fill the service */  
if (cl.call(srv)) { /* :) */ } else { /* :( */ }
```



BUILDING YOUR CODE

```
cmake_minimum_required(VERSION 2.8.3)
project(package_name)
find_package(catkin REQUIRED COMPONENTS roscpp std_msgs genmsg)
add_message_files(FILES custom_message.msg)
add_service_files(FILES custom_service.srv)
generate_messages(DEPENDENCIES std_msgs)
catkin_package()

include_directories(include ${catkin_INCLUDE_DIRS})
add_executable(executable_name src/source_code.cpp)
target_link_libraries(executable_name ${catkin_LIBRARIES})
add_dependencies(executable_name package_name_generate_messages_cpp)
```



BUILDING YOUR CODE

```
cmake_minimum_required(VERSION 2.8.3)
project(package_name)
find_package(catkin REQUIRED COMPONENTS roscpp std_msgs genmsg)
add_message_files(FILES custom_message.msg)
add_service_files(FILES custom_service.srv)
generate_messages(DEPENDENCIES std_msgs)
catkin_package()
```

This is what you have to
change depending on
your code!

```
include_directories(include ${catkin_INCLUDE_DIRS})
add_executable(executable_name src/source_code.cpp)
target_link_libraries(executable_name ${catkin_LIBRARIES})
add_dependencies(executable_name package_name_generate_messages_cpp)
```



BUILDING YOUR CODE

```
cmake_minimum_required(VERSION 2.8.3)
project(package_name)
find_package(catkin REQUIRED COMPONENTS roscpp std_msgs genmsg)
add_message_files(FILES custom_message.msg)
add_service_files(FILES custom_service.srv)
generate_messages(DEPENDENCIES std_msgs)
catkin_package()
```

Only if you have custom
messages!

```
include_directories(include ${catkin_INCLUDE_DIRS})
add_executable(executable_name src/source_code.cpp)
target_link_libraries(executable_name ${catkin_LIBRARIES})
add_dependencies(executable_name package_name_generate_messages_cpp)
```